

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA SAU ĐẠI HỌC



BÁO CÁO BÀI TẬP LỚN
CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG CỬA HÀNG ONLINE
THEO MÔ HÌNH MICROSERVICES

Giảng viên hướng dẫn

: TS. Kim Ngọc Bách

Lớp

: M25CQHT01-B

Nhóm

: Nhóm 1

01. Đinh Hòa Bình

MHV: B25CHHT005

02. Lê Hoàng

MHV: B25CHHT024

03. Vũ Mai Linh

MHV: NCS2025.6

Hà Nội – 2025

I. TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1. Đặt vấn đề

- Trong bối cảnh thương mại điện tử phát triển nóng, các hệ thống Monolithic (nguyên khối) cũ bộc lộ nhiều điểm yếu: khó mở rộng (scaling), khó bảo trì và rủi ro cao khi triển khai tính năng mới. Việc chuyển đổi sang kiến trúc Microservices là xu hướng tất yếu để giải quyết các bài toán về hiệu năng và tính linh hoạt..

1.2. Mục tiêu đề tài

- Xây dựng một hệ thống bán hàng trực tuyến (E-commerce) hoàn chỉnh bao gồm các phân hệ: Quản lý người dùng, Danh mục sản phẩm, Tìm kiếm, Giỏ hàng và Giao diện người dùng. Hệ thống phải đảm bảo:
 - Tính độc lập giữa các dịch vụ.
 - Khả năng chịu tải cao.
 - Bảo mật thông tin người dùng.
 - Sẵn sàng triển khai, mở rộng trên môi trường Container (Docker/Kubernetes).

II. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu chức năng

2.1.1. Quản lý người dùng (User)

Hệ thống cho phép quản lý thông tin khách hàng

- **Tạo mới người dùng:** Cho phép đăng ký người dùng mới với các thông tin: tên, email, số điện thoại.
- **Cập nhật thông tin:** Cho phép chỉnh sửa thông tin cá nhân (tên, email, mobile) dựa trên user_id.
- **Tra cứu thông tin:** Xem chi tiết thông tin một người dùng cụ thể hoặc lấy danh sách toàn bộ người dùng.

2.1.2. Quản lý sản phẩm (Product)

Hệ thống cung cấp thông tin sản phẩm và các chương trình khuyến mãi

- **Xem danh sách khuyến mãi (Deals):** Hiển thị danh sách các sản phẩm đang có deal (giới hạn hiển thị 50 sản phẩm).
- **Xem chi tiết sản phẩm:** Tìm kiếm và hiển thị thông tin chi tiết sản phẩm dựa trên mã SKU (Stock Keeping Unit).

- **Quản lý Sản phẩm:** CRUD các sản phẩm

2.1.3. Quản lý giỏ hàng (Shopping Cart)

Tính năng giỏ hàng với các nghiệp vụ phức tạp về đồng bộ dữ liệu.

- **Lấy thông tin giỏ hàng:** Xem giỏ hàng hiện tại của một khách hàng dựa trên customerId.
- **Thêm/Cập nhật giỏ hàng:**
 - Hệ thống kiểm tra giỏ hàng cũ của khách trong Redis.
 - **Cơ chế Merge (Gộp):** Nếu sản phẩm đã tồn tại trong giỏ, hệ thống sẽ cộng dồn số lượng (quantity). Nếu chưa, sản phẩm mới sẽ được thêm vào danh sách.
- **Tính tổng tiền (Auto Calculation):** Hệ thống tự động tính lại tổng giá trị đơn hàng (total) dựa trên đơn giá và số lượng mỗi khi giỏ hàng được cập nhật.

2.1.4. Tìm kiếm (Search)

- **Tìm kiếm thông tin:** Hỗ trợ tìm kiếm dữ liệu (sản phẩm, danh mục) ở thanh tìm kiếm

2.1.5. Xác thực người dùng (Authentication):

Cung cấp tính năng xác thực người dùng:

- **Đăng nhập:** Hệ thống cho phép người dùng đăng nhập với tài khoản/mật khẩu đã được tạo từ trước, sau khi đăng nhập thì sẽ có token (jwt) để đảm bảo tính năng
- **Đăng xuất:** Hệ thống cho phép người dùng đăng xuất khỏi hệ thống, clear token đã được tạo ra từ phiên trước.

2.2. Thiết kế Microservices

Để chuyển hóa từ các yêu cầu nghiệp vụ sang kiến trúc Microservices cụ thể, nhóm thực hiện đã áp dụng tư duy **Domain-Driven Design (DDD)**. Quy trình phân tách được thực hiện theo nguyên tắc: "**Mỗi Domain nghiệp vụ sở hữu một cấu trúc dữ liệu riêng biệt, và Service được sinh ra để quản lý vòng đời của dữ liệu đó**".

Quy trình thiết kế cụ thể bao gồm 3 bước:

Bước 1: Phân tích & Phân loại dữ liệu (Data Analysis & Classification)

Trước khi viết bất kỳ dòng code nào, chúng tôi phân tích đặc thù của các luồng dữ liệu trong hệ thống để chọn công nghệ lưu trữ (Database) phù hợp nhất (Polyglot Persistence):

Bước 2: Xác định Ngữ cảnh giới hạn (Bounded Context)

Dựa trên các domain nghiệp vụ đã phân tích, chúng tôi vẽ ra các ranh giới (Boundaries) với tiêu chí khác loại DB và độc lập tương quan liên quan đến dữ liệu.

Bước 3: Xây dựng Service bao quanh (Service Wrapping)

Service được xây dựng đóng vai trò là "người gác cổng" (Gatekeeper) cho Database.

- Các Service bên ngoài *không được phép* truy cập trực tiếp vào DB của Service khác. Ví dụ: Cart Service muốn lấy tên người dùng thì không được Query vào PostgreSQL, mà phải gọi API sang Users Service.
- Điều này đảm bảo tính đóng gói (Encapsulation) và giảm sự phụ thuộc (Decoupling) giữa các phân hệ.

2.3. Phân tích & Phân loại dữ liệu:

2.3.1. Thiết kế cho domain User

Lưu trữ dữ liệu có cấu trúc chặt chẽ.

- Bảng **users**:
 - id (PK) uuid => mục đích có thể lưu phân tán
 - name: varchar(50)
 - email (Unique)
 - hashed_password
 - salt
 - mobile
- ⇒ Đánh giá dữ liệu: quan trọng, cần lưu trữ và có tính toàn vẹn cao, tần xuất cập nhật thấp trên người dùng, tần xuất đọc thấp
- ⇒ Chọn lưu trữ trên hệ CSDL **PostgreSQL**
- Bảng **user_tokens**:
 - id (PK)
 - user_id
 - token
 - refresh_token
 - last_updated
- ⇒ Đánh giá dữ liệu: quan trọng, có thể mất dữ liệu, tần xuất cập nhật ít, tần xuất đọc nhiều
- ⇒ Chọn lưu trữ trên **Redis** để đảm bảo hiệu năng đọc

2.3.2. Thiết kế cho domain Product

- Bảng **product**

- id
- department
- category
- thumbnail
- image
- title
- description
- shortDescription
- price
- currency
- rating
- attributes
- secondaryAttributes
- variant
- lastUpdated

⇒ Đánh giá dữ liệu: quan trọng, cần lưu trữ và có tính toàn vẹn cao, tần xuất cập nhật thấp trên người dùng, tần xuất đọc cao, query filter danh sách theo các điều kiện

⇒ Chọn lưu trữ trên **MongoDB** để đảm bảo hiệu năng đọc.

- Bảng **deals**

- dealId
- productId
- variantSku
- department
- thumbnail
- image
- title
- description
- shortDescription
- price
- currency
- rating
- lastUpdated

⇒ Đánh giá: giống product

⇒ Chọn lưu trữ trên **MongoDB**

2.3.3. Thiết kế cho domain Cart

- Bảng **Cart**:
 - customerId
 - items: List
 - productId
 - sku
 - title
 - quantity
 - price
 - currency
 - total
 - currency
- Đánh giá dữ liệu: quan trọng, có thể mất dữ liệu, tần xuất cập nhật nhiều, tần xuất đọc trung bình
- Chọn lưu trữ trên **Redis** để đảm bảo hiệu năng đọc/ghi

2.4. Xác định Ngữ cảnh giới hạn (Bounded Context)

Dựa trên các DB đã chọn, chúng tôi vẽ ra các ranh giới (Boundaries)

- *Boundary 1 (User Context)*: Bao quanh PostgreSQL.
- *Boundary 2 (Product Context)*: Bao quanh MongoDB.
- *Boundary 3 (Cart Context)*: Bao quanh Redis.
- *Boundary 4 (Token Context)*: Bao quanh Redis.

2.5. Xác định các Services:

2.5.1. Cart service

- Chức năng:
 - Lấy giỏ hàng theo mã khách hàng
 - Thêm hàng vào giỏ hàng của khách hàng
- Thiết kế API:
 - GET /cart: trả lại giỏ hàng của người dùng (yêu cầu token của Authorization trên header)
 - POST /cart: nhận vào payload cart để cập nhật vào giỏ hàng cũ (yêu cầu token của Authorization trên header)

2.5.2. Product service

- Chức năng:
 - Lấy danh sách sản phẩm
 - Lấy danh sách sản phẩm theo danh mục
 - Lấy chi tiết sản phẩm
 - Lấy top danh sách khuyến mãi (deals)
- Thiết kế API:
 - GET /products/{pageNo}/{pageSize}: danh sách sản phẩm theo phân trang
 - GET /products/{categoryId}/{pageNo}/{pageSize}: trả lại danh sách sản phẩm có phân trang theo mã danh mục categoryId
 - GET /product/sku/{id}: Trả lại chi tiết sản phẩm theo mã
 - GET /deals: lấy danh sách các sản phẩm khuyến mãi (limit 50)

2.5.3. Search service

- Chức năng:
 - Tìm kiếm sản phẩm
 - Gợi ý sản phẩm (suggestion khi typing vào ô search)
- Thiết kế API:
 - POST /search/{keyword}/{pageNo}/{pageSize}: Trả lại danh sách sản phẩm theo phân trang sắp xếp theo độ chính xác tìm kiếm (áp dụng search engine của Elasticsearch)
 - POST /search/{keyword}: lấy danh sách sản phẩm gợi ý theo từ khóa (top 10)

2.5.4. User service

- Chức năng:
 - Tạo mới/ đăng ký user
 - Cập nhật thông tin user (bao gồm cả đổi mật khẩu, quên mật khẩu)
 - Lấy thông tin user theo id
- Thiết kế API:
 - POST /users: Nhận vào payload data gồm name, email, mobile, password để tạo user mới.
 - POST /users/{user_id}: Cập nhật thông tin dựa trên user_id (yêu cầu token của Authorization trên header)

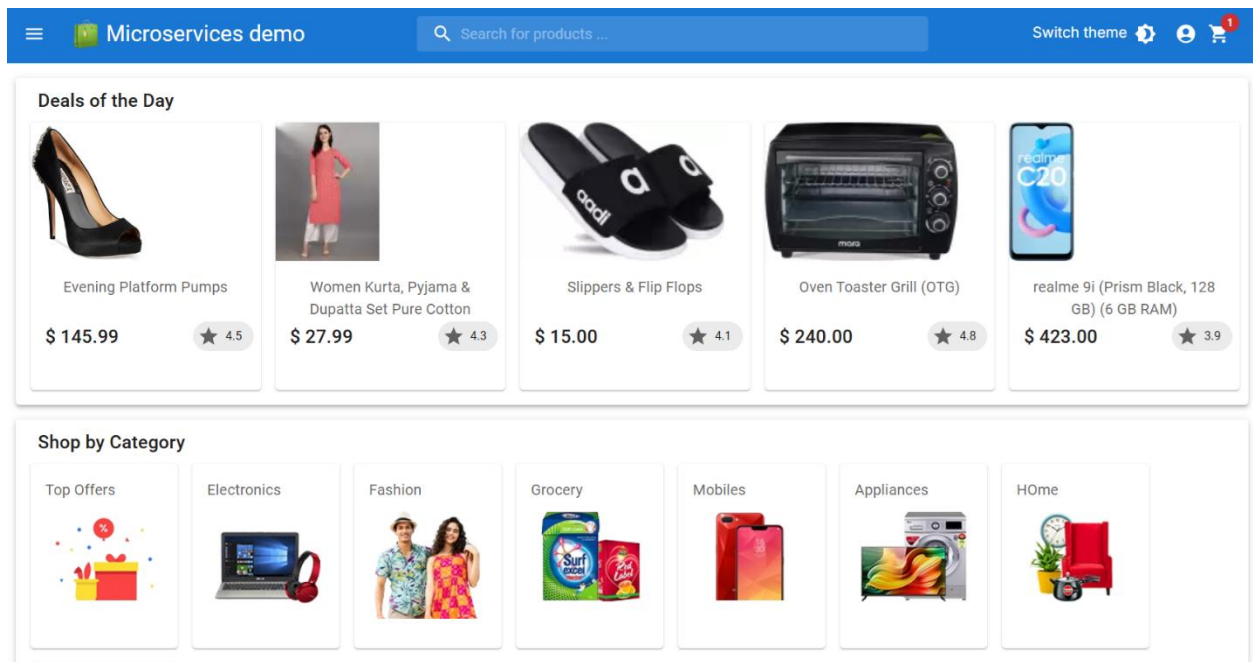
- GET /users/{user_id}: Lấy chi tiết user theo user_id (yêu cầu token của Authorization trên header)

2.5.5. Authentication Service

- Chức năng:
 - Đăng nhập
 - Đăng xuất
 - Refresh token
- Thiết kế API:
 - POST /user/login: Nhận vào payload data gồm user, password để tạo user mới.
 - POST /user/logout: Đăng xuất (yêu cầu token của Authorization trên header)
 - POST /user/refresh_token: Lấy chi tiết user theo user_id (yêu cầu token của Authorization trên header)

2.5.6. Frontend service

- Màn hình/chức năng:
 - Trang chủ
 - Đăng ký / Đăng nhập / Đăng xuất
 - Giỏ hàng
 - Thanh toán

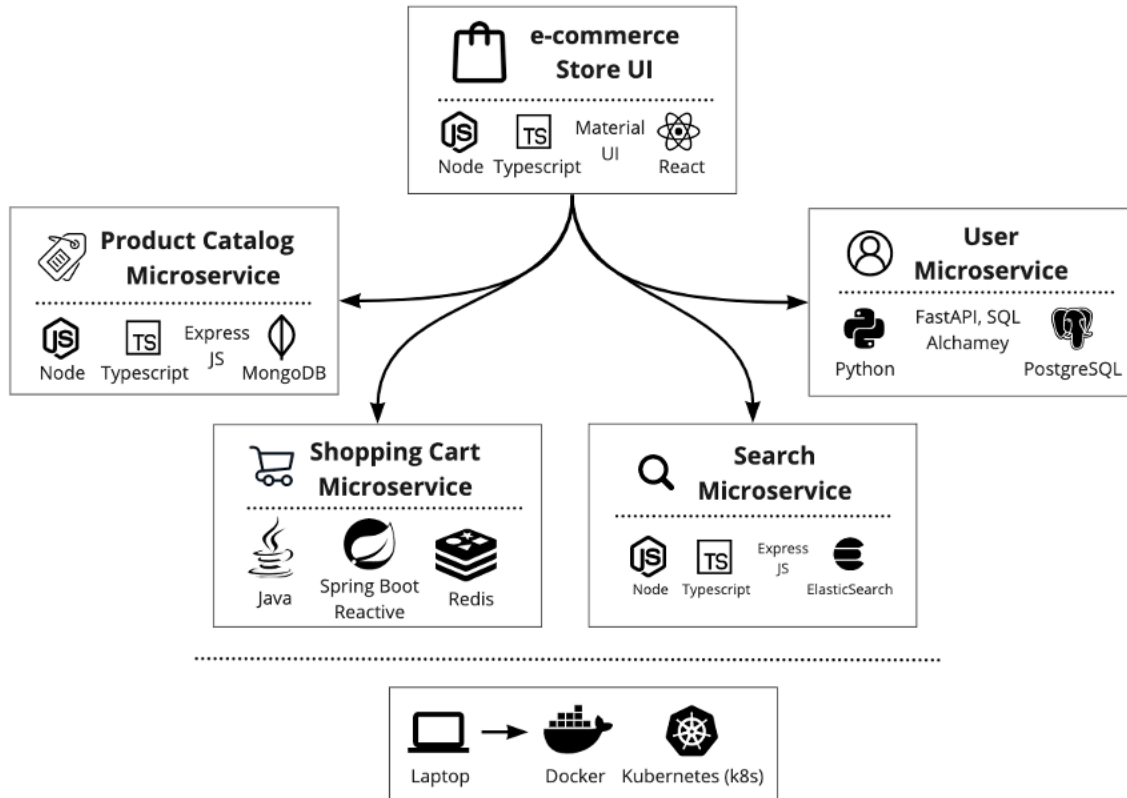


III. Kiến trúc hệ thống và phân hệ sử dụng

3.1. Kiến trúc tổng thể

Hệ thống áp dụng kiến trúc **Polyglot Microservices**, nghĩa là mỗi dịch vụ được viết bằng ngôn ngữ lập trình và sử dụng công nghệ lưu trữ dữ liệu phù hợp nhất với nghiệp vụ của nó, thay vì ép buộc sử dụng một công nghệ duy nhất cho toàn bộ hệ thống.

- **Giao thức giao tiếp:** Các service giao tiếp với nhau qua HTTP RESTful API.
- **Triển khai:** Đóng gói bằng Docker Containers và triển khai trên nền tảng Kubernetes (K8s).



3.2. Tech Stack

Phân hệ (Microservice)	Ngôn ngữ / Framework	Cơ sở dữ liệu	Lý do lựa chọn
Store UI	ReactJS, TypeScript, Material UI	N/A	Cộng đồng người dùng lớn, nhiều bộ giao diện UI sẵn hỗ trợ. Chạy nhẹ trên docker.
Product Service	NodeJS, ExpressJS	MongoDB	Dữ liệu sản phẩm có cấu trúc linh động như thuộc tính của sản phẩm (JSON), phù hợp NoSQL document store.
Cart Service	Java, Spring Boot (WebFlux)	Redis	Cần tốc độ đọc/ghi cực nhanh cho dữ liệu tạm thời (Session), ngôn ngữ Java phổ biến và dễ kiểm soát.
User Service	Python, FastAPI	PostgreSQL	Xử lý nhanh, code gọn gàng, phù hợp dữ liệu quan hệ (Relational Data).

Search Service	NodeJS	ElasticSearch	Tối ưu hóa cho việc tìm kiếm full-text, gợi ý từ khóa. Có thể đồng bộ tự động sản phẩm từ MongoDB sang ES.
----------------	--------	---------------	--

IV. TRIỂN KHAI HẠ TẦNG

Đây là phần trọng tâm về kỹ thuật vận hành (DevOps).

4.1. Containerization (Docker)

Mỗi service đều có Dockerfile riêng biệt để đóng gói môi trường chạy.

- **users:** Python Base Image.
- **cart:** OpenJDK Image (Multi-stage build).
- **store-ui:** Node Build -> Nginx Serve (Production optimize).
- **products:** nodejs base Image
- **search:** nodejs base Image

	Name 	Tag	Image ID
	store-ui	latest	ff4f29f13e40
	cart	latest	77b50387f631
	products	latest	74c5f8b60d87
	users	latest	dfae6f0aeae5
	search	latest	976195da8373

4.2. Kubernetes Orchestration (Production Ready)

Hệ thống sử dụng Kustomize để quản lý cấu hình cho các môi trường (Dev/Prod).

4.2.1. Quản lý trạng thái (StatefulSets vs Deployments)

- **Application (Stateless):** Sử dụng Deployment (Cart, User, UI) để dễ dàng scale số lượng Pods.

- **Database (Stateful):** Sử dụng StatefulSet kết hợp PersistentVolumeClaim (PVC) cho MongoDB, PostgreSQL, Redis để đảm bảo dữ liệu không bị mất khi Pod khởi động lại.

4.2.2. Cơ chế tự phục hồi (Self-Healing)

Tích hợp **Probes** vào deployment.yaml để K8s tự động giám sát:

- livenessProbe: Restart container nếu ứng dụng bị treo (Deadlock).
- readinessProbe: Chỉ điều hướng traffic vào khi ứng dụng đã khởi động xong hoàn toàn.

4.2.3. Điều hướng Traffic (Ingress Controller)

Thay vì dùng NodePort sơ khai, hệ thống sử dụng **Ingress Controller** làm cổng vào duy nhất (API Gateway Pattern):

- Domain: api.store-demo.vn
 - Route /api/cart/* -> cart-service
 - Route /api/users/* -> users-service
- Domain: store-demo.vn
 - Route / -> store-ui

4.3. Giám sát hệ thống (Observability)

Để vận hành ổn định, hệ thống đề xuất tích hợp:

- **Logging:** EFK Stack (Elasticsearch - Fluentd - Kibana) để gom log tập trung.
- **Tracing:** Jaeger để truy vết request qua các microservices.

V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

- Đã phân tích và hiểu rõ được phương pháp phân tích thiết kế hệ thống triển khai kiến trúc microservices.
- Đã phát triển thành công mô hình nghiệp vụ các services cơ bản đã phân tích.
- Đã cài đặt được ứng dụng chạy đáp ứng được các yêu cầu cơ bản đã đề ra.

5.2. Hạn chế của đề tài

- Chưa phát triển hết nghiệp vụ ứng dụng của Search Service, User service và Token Service.

- Chưa tích hợp hoàn toàn đúng mô hình trên K8S, tích hợp ingress làm API Gateway, chưa tích hợp liveness/readiness, chưa tích hợp giám sát hệ thống
- Chưa có giao diện hiển thị sách theo danh mục, phân trang danh sách sản phẩm, thanh toán và trả hàng/hoàn phí
- Chưa có giao diện để admin quản lý danh mục, sản phẩm, doanh thu, đơn hàng, ...

5.3. Hướng phát triển trong tương lai

- Hoàn thiện nốt các hạn chế của đề tài.
- Tích hợp Service Mesh (Istio) để quản lý traffic tốt hơn.
- Bổ sung module Payment (Thanh toán) kết nối với cổng thanh toán thực tế (Stripe/PayPal).
- Sử dụng Message Queue (Kafka) cho xử lý đơn hàng bất đồng bộ.

VI. PHỤ LỤC

- **Hướng dẫn cài đặt:**
Tham khảo trong tài liệu trong thư mục repo: Hướng dẫn cài đặt.pdf
- **Link source code:**
https://github.com/hoangl/CH_HTTP_2025.1/tree/main/e-commerce-microservices